

Meme Stock Scanner

Pseudocode Specification for Walk-Forward Testing

Quant research notes

September 18, 2026

1 Module 0. Constants

```
MODULE config

# ---- credentials, from environment, never hardcoded ----
CONST ALPACA_KEY      = env("APCA_API_KEY_ID")
CONST ALPACA_SECRET   = env("APCA_API_SECRET_KEY")

# ---- endpoints ----
CONST EP_BARS         = "https://data.alpaca.markets/v2/stocks/bars"
CONST EP_ASSETS       = "https://api.alpaca.markets/v2/assets"
CONST EP_OPT_C        = "https://api.alpaca.markets/v2/options/contracts"
CONST EP_OPT_B        = "https://data.alpaca.markets/v1beta1/options/bars"
CONST EP_FINRA        = "https://cdn.finra.org/equity/otcmarket/biweekly/shrt{YYYYMMDD}.csv"
CONST EP_ARCTIC       = "https://arctic-shift.photon-reddit.com/api/comments/search"
CONST EP_LLM          = LLM_BASE + "/chat/completions"      # OpenRouter, OpenAI shape
CONST EP_STWITS       = "https://api.stocktwits.com/api/2/streams/symbol/{sym}.json"
CONST EP_EDGAR        = "https://data.sec.gov/api/xbrl/companyconcept"

# ---- language model. OpenRouter, DeepSeek. Chosen for cost. ----
CONST LLM_MODEL = env("DEEPSEEK_MODEL", "deepseek/deepseek-v4-flash")
CONST LLM_BASE  = env("DEEPSEEK_BASE_URL", "https://openrouter.ai/api/v1")
CONST LLM_KEY   = env("DEEPSEEK_API_KEY")
CONST LLM_TEMP  = 0.1      # as configured. Note: 0.0 would be strictly better
                        # for reproducibility. A score that shifts on re-run
                        # makes a backtest non-deterministic. Consider 0.0.

CONST LLM_MAXTOK = 250
CONST LLM_CACHE  = CACHE / "llm_context.json"      # keyed (ticker, date)
CONST LLM_ENABLED = false # v1 runs WITHOUT the LLM. Default off.

# ---- FIXED for the whole study. Never re-fitted per fold. ----
CONST RVOL_MIN    = 2.0    # sweep 1.75..3.0 gave Sharpe 0.83..0.91, best/median
                        # 1.54x, a plateau not a peak. Above 3.5 -> 0.40..0.60
CONST EXT_CAP     = 0.50   # ablation: none $434,670 | 0.50 $519,858 | 0.30 and
                        # 0.15 both lower. Ignition days gap >30% over MA20

CONST MOM5_MIN    = 0.05
CONST HARD_STOP   = 0.08  # daily ATR is 4..8%, tighter is hit by normal moves.
                        # 12% tested better (Sharpe 1.18 vs 1.07) but the
                        # $108k gap is smaller than one trade (LAES $115k),
                        # so it is inside noise. Keep 8% to avoid fitting.

CONST EARLY_DAYS  = 2
CONST EARLY_MFE   = 0.03  # ablation +$21,491, winners >50% fell only 37 -> 35.
                        # 5% after 3d lost $41,692. 10% after 3d lost more.

CONST ATR_MULT    = 3.0    # 2 ATR sits inside the noise band of a 5%/day stock
CONST MA_EXIT     = 10
CONST MAX_HOLD    = 60     # observed winners held 8..37 sessions
CONST POS_SIZE    = 10000
CONST MAX_POS     = 10     # 10 x $10k = full $100k, no compounding
CONST CAPITAL     = 100000
CONST LIQ_FLOOR   = 1000000
CONST LIQ_CEIL    = 300000000
CONST PRICE_MIN   = 0.20
CONST SI_LAG_DAYS = 12     # FINRA disseminates ~8 business days after settlement
CONST MAX_PER_THEME = 3    # Module 15. 10 crypto miners is ONE position
CONST MAX_CORR_PAIRWISE = 0.70 # Module 15, on 60 day returns through t-1
CONST ENTRY_LIMIT_PCT = 0.03 # Module 19, marketable limit above last close
CONST MAX_CHURN    = 0.30  # Module 21, daily universe turnover target
CONST TZ           = "America/New_York" # Module 3. ALL dates are ET.

# ---- FITTED on each training window. Re-fitted every fold. ----
VAR N_UNIVERSE, ATR_Q, RET20_Q      # PERCENTILES, not absolute levels.
                        # Absolute levels make universe size drift
                        # 8x across the test period. See Module 6.
```

```

VAR    W_ATR, W_DD, W_VB, W_SOC, W_LVL, W_LLM, W_OPT

# ---- PRIMITIVES assumed available. Declared so nothing is ambiguous. ----
# z(x)          cross-sectional z-score across all symbols on ONE day.
#               z(x)[s,d] = (x[s,d] - mean(x[:,d])) / std(x[:,d])
#               Never looks forward. Always safe.
# pct(x, q)     the q-th percentile of x over a TRAILING window only.
#               Computing it over the full sample is look-ahead. See Module 12.3.
# top_n(L, n)   the n highest-scoring entries of L, ties broken by symbol.
# corr(a, b)    Pearson correlation on the given slice.
# spearman()    rank correlation, used for the information coefficient.
# All other lower-case calls (mean, max, sqrt, median, sleep, parse_csv,
# sha256, regex_findall) are ordinary library functions.

# ---- NEVER fitted, structurally forbidden -----
# The symbol list itself. No hand-picked names from any source, ever,
# including a language model. This is the error that invalidated v1.
END MODULE

```

2 Module 1. Data ingestion

```

MODULE ingest

FUNCTION fetch_symbol_master()
    r = GET EP_ASSETS {status:"active", asset_class:"us_equity"}
    headers {APCA-API-KEY-ID: ALPACA_KEY, APCA-API-SECRET-KEY: ALPACA_SECRET}
    keep = []
    FOR a IN r:
        IF NOT a.tradable: CONTINUE
        IF a.exchange NOT IN {NYSE, NASDAQ, AMEX, ARCA, BATS}: CONTINUE
        IF NOT is_alpha(a.symbol) OR len(a.symbol) > 5: CONTINUE
        IF any_substring(upper(a.name), [ETF, ETN, FUND, TRUST, INDEX, ISHARES,
            PROSHARES, DIREXION, INVESCO, SPDR, VANGUARD, BULL, BEAR, "2X",
            "3X", ULTRA, LEVERAGED, NOTES, PREFERRED, WARRANT, DEPOSITARY,
            UNIT, "ACQUISITION CORP"]): CONTINUE
        keep.APPEND(a.symbol)
    RETURN keep # approx 5,033 US common stocks

# DO NOT USE THIS ALONE. It returns only currently listed symbols, so it
# is a SURVIVOR list, not a universe. 27 of the original 82 names are
# already absent (33%): APE ATNF BBY BBIG BIGC CARM COMS CRKN EXPR FFIE
# GREE HOLO HUDI IRNT MARK MMAT MULN NKLA PROG RDBX RIDE SAVA SDIG SPRT
# TRKA TUP WISH
# status="all" recovers only 9 of those 27, and each has fallen to OTC.
# MODULE 22 REPLACES THIS FUNCTION. It is retained only for asset
# metadata (exchange, name) on symbols that still exist.

FUNCTION fetch_bars(symbols, timeframe, start, adjustment)
    out = []
    FOR chunk IN batches(symbols, 100): # 100 symbols per request
        token = null
        WHILE true:
            r = GET EP_BARS {symbols: chunk, timeframe: timeframe,
                start: start, end: today, feed: "sip",
                adjustment: adjustment, limit: 10000,
                page_token: token}
            out.APPEND(r.bars)
            token = r.next_page_token
            IF token IS null: BREAK
    RETURN out

# TWO SEPARATE PRICE PANELS ARE MANDATORY.
px_split = fetch_bars(master, "1Day", "2016-01-01", "split") # for RETURNS
px_raw    = fetch_bars(master, "1Day", "2016-01-01", "raw")    # for PRICE LEVEL
px_15m    = fetch_bars(watchlist_only, "15Min", "2016-01-01", "split")

# Reason: Alpaca adjusts backward. A 1-for-100 reverse split in 2023 multiplies
# the shown 2021 price by 100. One symbol showed $2.98 trillion in 2016. 463
# symbols showed >$1000 while trading as penny stocks, 7.8% of all events.
# Filtering on adjusted price level therefore reads a FUTURE corporate action.
# This leak is currently unfixed in the existing code base.
#
# Reason for start=2016: the screen uses drawdown from the all-time high. If
# history starts in 2021 that high is truncated and the measure is meaningless.
#
# TIME ZONE. Alpaca returns RFC3339 in UTC. Convert to America/New_York FIRST,
# then take the calendar date, THEN normalise. Taking the date from naive UTC
# pushes every bar timestamped after 20:00 ET into the following day. This is

```

```

# the same class of defect as gate_1 and it is silent.

FUNCTION fetch_short_interest()
rows = []
FOR d IN candidate_settlement_dates(2021-01 .. today):
    # settlements fall near the 15th and the last business day.
    # probe d-2 .. d+2 to locate the published file.
    url = EP_FINRA.format(d)
    r = GET url headers {User-Agent: "<a browser UA string>"}
    # the default python-requests agent is rejected with HTTP 403
    IF r.status != 200 OR len(r.body) < 50000: CONTINUE
    t = parse_csv(r.body, delimiter="|")
    FOR row IN t:
        rows.APPEND({ticker: row.symbolCode,
                      short_int: row.currentShortPositionQuantity,
                      prev_si: row.previousShortPositionQuantity,
                      adv: row.averageDailyVolumeQuantity,
                      dtc: row.daysToCoverQuantity,
                      settle: d,
                      usable_from: d + SI_LAG_DAYS}) # THE CRITICAL LINE
RETURN rows # 522,237 rows, 5,001 symbols, 131 periods, 2021-01 onward

FUNCTION fetch_social(day d)
docs = []; after = epoch(d); before = epoch(d) + 86400
WHILE true:
    r = GET EP_ARCTIC {subreddit: "wallstreetbets", after: after,
                      before: before, limit: 100}
    IF r.status IN {422, 429}: sleep(30); retry up to 5; CONTINUE
    IF empty(r): BREAK
    FOR doc IN r:
        docs.APPEND({doc_id: doc.created_utc, author_hash: sha256(doc.author),
                      score: doc.ups, num_comments: doc.parent_id,
                      body_text: doc.body}) # STORE FULL TEXT
    after = max(created_utc IN r)
RETURN docs

# The previous version stored only a count of ticker mentions per day.
# A count cannot separate 100 posts by 100 people from 100 posts by 1
# account, cannot separate a thesis from a warning, and cannot see whether
# a catalyst is named. Those three are the entire reason to add an LLM.
#
# BLOCKED, do not attempt: reddit.com public JSON returns 403 to automated
# agents. api.stocktwits.com returns 403 and 429. Pushshift is discontinued.
# Live production feed with no history: apewisdom.io/api/v1.0/filter/
# wallstreetbets/page/{n} (free, no key, cannot backtest)

FUNCTION fetch_borrow_iborrowdesk(symbol s)
r = GET "https://iborrowdesk.com/api/ticker/" + s
headers {User-Agent: "<browser UA>", Accept: "application/json"}
RETURN {fee_pct: r.fee, available: r.available, updated: r.updated}
# WORKS with a browser User-Agent. Returns 403 with the default agent.
# LIMITATION: a SNAPSHOT, not history. Cannot be backtested. This is the
# only free path to borrow data, and only going forward. Start recording
# today; see Module 11.2.

FUNCTION fetch_options(ticker s, day d)
spot = px_raw[s, d]
c = GET EP_OPT_C {underlying_symbols: s, type: "call",
                  strike_price_gte: 1.10 * spot,
                  expiration_date_gte: d, expiration_date_lte: d + 90}
b = GET EP_OPT_B {symbols: occ_symbols(c), timeframe: "1Day"}
RETURN sum(bar.volume FOR bar IN b WHERE bar.date == d)
# Coverage begins 2024-01-22. No earlier history exists at this price point.
END MODULE

```

3 Module 2. Quality gates

Each gate exists because a specific defect was found and measured. Fail loudly.

```

MODULE gates

FUNCTION gate_1_timestamps(left, right)
# DEFECT FOUND: the price panel carried 05:00:00 while external sources used
# 00:00:00. The join produced ZERO overlapping days and every social feature
# silently became zero. The conclusion "social data is useless" was an
# artifact of the join, not a finding.

```

```

left.index = normalize_to_calendar_date(tz_naive(left.index))
right.index = normalize_to_calendar_date(tz_naive(right.index))
joined = left.join(right)
overlap = count(rows where right columns are not null)
PRINT "coverage:", overlap, "of", len(left)
ASSERT overlap > 0.5 * expected_days # never proceed on zero coverage
RETURN joined

FUNCTION gate_2_corporate_actions(row)
# DEFECT FOUND: GPOR showed a 52,648% 20-day gain, AQB showed 367,817%.
# These are bankruptcy reorganisations that reused the ticker, leaving a
# discontinuity in the adjusted series. Removes 11% of detected events.
IF row.adv20_usd IS null OR row.adv20_usd < 10000: RETURN false
IF row.atr_pct IS null OR row.atr_pct <= 0.005
OR row.atr_pct >= 1.00: RETURN false
IF row.price_raw < 0.10 OR row.price_raw > 500: RETURN false
IF abs(row.ret_20d) > 5.00: RETURN false
RETURN true

FUNCTION gate_3_ticker_extraction(text)
# DEFECT FOUND: upper(whole sentence) turned "will be" into ticker BE and
# "any" into ANY. 46% of all mentions were ordinary English words.
cands = regex_findall(text, "\\$[A-Z]{1,5}\\b | \\b[A-Z]{2,5}\\b")
RETURN [c FOR c IN cands IF c IN master AND c NOT IN blocklist]

FUNCTION build_blocklist(all_docs)
# A real ticker spikes on news. An English word is flat over time.
FOR tok IN all_tokens:
burst[tok] = max(daily_count[tok]) / median(daily_count[tok])
RETURN {tok WHERE burst[tok] < 10 AND total_count[tok] > threshold}
# separates cleanly at burst = 10
#
# A first attempt correlated each token with the daily total and was WRONG:
# it flagged GME and AMC because those two DRIVE the total. The burst
# statistic does not have that failure.

FUNCTION decontaminate(mentions_for_ticker, english_word_proxy)
# residual removal: strip the component explained by overall chatter volume
beta = regress(mentions_for_ticker ON english_word_proxy).slope
RETURN mentions_for_ticker - beta * english_word_proxy

FUNCTION gate_4_null_policy(value, policy)
# DEFECT FOUND: a short-interest gate returned false when data was absent.
# That silently deleted 2021 through 2024, left 84 trades, and reported a
# loss of $389. The bug looked exactly like a strategy result.
IF value IS null:
PRINT "null count for this gate:", counter
IF policy == "allow": RETURN true
IF policy == "block": RETURN false
FAIL "policy must be declared explicitly"
RETURN evaluate(value)

FUNCTION gate_5_feature_sign(feature, horizon)
# DEFECT FOUND: social level entered the composite with a POSITIVE weight
# while its measured IC is -0.054. The options term used acceleration
# (IC ~ 0) instead of level (IC +0.061). Fixing it moved expectancy from
# +1.54% to +2.10%.
ic = information_coefficient(feature, horizon, universe = NEUTRAL)
ASSERT sign(assigned_weight) == sign(ic)
ASSERT abs(ic) > 0.01 # otherwise the feature contributes noise
RETURN ic

FUNCTION gate_6_solveny(equity)
# DEFECT FOUND: fixed $10k sizing kept trading after cumulative profit hit
# -$878,000 on $100,000 of capital. The reported maxDD of -655% is
# meaningless. Always report dollar profit beside any percentage drawdown.
IF equity < POS_SIZE: RETURN "HALT_NEW_ENTRIES"
RETURN "OK"
END MODULE

```

4 Module 3. Features

MODULE features

```

FUNCTION build_features(symbol s, day t)
# EVERY line below reads t-1 or earlier. No value from day t appears here.
f = {}

# liquidity
f.adv20_usd = mean(close_split[t-20:t] * volume[t-20:t])
f.adv20_share = mean(volume[t-20:t])

# volatility
tr = max(high - low, abs(high - prev_close), abs(low - prev_close))
f.ATR14 = mean(tr[t-14:t]) # used by the trailing stop, Module 8
f.ATR20 = mean(tr[t-20:t]) # used by the volatility screen
f.ATR_pct = f.ATR20 / close_split[t-1]

# trend and position. All RATIOS, so immune to split adjustment.
f.ma20 = mean(close_split[t-20:t])
f.ma50 = mean(close_split[t-50:t])
f.ATH = cummax(close_split[2016-01 : t-1])
f.dd_from_ATH = close_split[t-1] / f.ATH - 1
f.close = close_split[t] # the ONLY value from day t. Used by
# the Module 7 trigger at the close,
# never by the Module 6 universe.

f.ret_5d = close_split[t-1] / close_split[t-6] - 1
f.ret_20d = close_split[t-1] / close_split[t-21] - 1
f.ret_60d = close_split[t-1] / close_split[t-61] - 1
f.pct_vs_ma20 = close_split[t-1] / f.ma20 - 1

# volume behaviour
f.rvol = volume[t] / mean(volume[t-20:t]) # evaluated at close t
f.vol_build = mean(volume[t-20:t]) / mean(volume[t-60:t])

# price LEVEL uses the RAW panel, never the adjusted one
f.price_raw = close_raw[t-1]

# short interest, publication lagged
si = last row WHERE si.usable_from <= t
f.dtc = si.dtc
f.si_change = si.short_int / si.prev_si - 1

# history. An event is only KNOWABLE at its peak, so it becomes usable at
# peak_date + 1, never at the day the run began.
f.prior_memes = count(events WHERE event.peak_date + 1 <= t)

# social and options, Modules 4 and 3b
soc = aggregate_social(s, t-1) # Module 4. Fields may be null when
# a source failed; see Module 17.

f.soc_level = soc.soc_level
f.soc_accel = soc.soc_accel
f.soc_authors = soc.soc_authors
f.concentration = soc.concentration
f.st_bull_ratio = soc.st_bull_ratio
f.llm_conviction = soc.llm_conviction
f.llm_catalyst = soc.llm_catalyst
f.llm_coordination = soc.llm_coordination
f.llm_novelty = soc.llm_novelty
f.otm_call_level = mean(otm_call_volume[t-5:t]) # LEVEL not acceleration
RETURN f

FUNCTION detect_meme_events(symbol s)
# Used to build prior_memes and to build the label in Module 9.
events = []; i = 60
WHILE i < len(close) - 1:
    w = close[i+1 : i+21] # forward 20 sessions
    IF max(w) / close[i] - 1 >= 0.50:
        pk = argmax(w)
        events.APPEND({ignition: i, peak: pk, gain: max(w)/close[i] - 1,
            knowable_from: pk + 1}) # NOT from i
        # no overlapping events
        i = pk + 1
    ELSE: i = i + 1
RETURN events
END MODULE

```

5 Module 4. Language model layer

```

MODULE llm

# =====
# WHAT THE MODEL MAY AND MAY NOT DO

```

```

#
# ALLOWED: read social text that already exists and convert it to numbers.
# This is a text-processing job.
# FORBIDDEN: generate a candidate list, name tickers from memory, or answer
# any question about what happened after its training cutoff.
#
# This is the central lesson of the project. The original 82 name universe came
# from asking a model to name potential meme stocks. It already knew the answer.
# That single call invalidated every backtest built on top of it.
# A model may PROCESS the past. It must never RECALL the past.
#
# ARCHITECTURAL RULE, from the v1 design:
# the deterministic composite is computed FIRST and IN FULL.
# the LLM is an OVERLAY on the surviving top-N names only.
# it never sees the other 5,000 symbols and never changes an entry or exit.
# LLM_ENABLED defaults to FALSE. v1 runs and is evaluated without it.
# =====

# ---- STEP 1. FREE PRE-LABELLED SENTIMENT, BEFORE SPENDING ANY TOKEN ----
# StockTwits already ships a Bullish or Bearish label on every message.
# Count the ratio directly. No model call is needed for stance.
FUNCTION free_sentiment(symbol s, day d)
  r = GET EP_STWITS.format(sym=s)
  msgs = [m FOR m IN r.messages IF date(m.created_at) == d]
  bull = count(m WHERE m.entities.sentiment.basic == "Bullish")
  bear = count(m WHERE m.entities.sentiment.basic == "Bearish")
  RETURN {st_count: len(msgs),
          st_bull_ratio: bull / max(bull + bear, 1)}
# NOTE: this endpoint returns 403 and 429 intermittently. Treat as
# best-effort. Apply gate_4 with policy "allow" so a null does not
# silently delete history, which is defect 4.

# ---- STEP 2. LLM ONLY FOR WHAT COUNTING CANNOT PRODUCE -----
# Counting gives volume and stance. It cannot give narrative or catalyst.
# Those two are the only reasons to spend a token.
CONST SYS_PROMPT = """
You are a text classifier for financial social media. You will be given recent
posts about one ticker from one day. Return only a JSON object, no prose.
Fields:
  conviction integer 0 to 3, the strongest level present in the posts.
              0 = passing mentions only
              1 = opinion with no support
              2 = specific reasoning or a price target
              3 = authors state positions held, with size or entry
  catalyst short phrase naming a specific expected event, for example
            earnings, FDA decision, short squeeze, merger vote, lockup
            expiry, or null if none is named
  coordination true when the posts read as coordinated promotion rather than
            independent opinion: repeated identical slogans, an explicit
            call to buy together, obvious copy paste
  novelty true when this reads as a new thesis rather than continuation
            of an established one
  time_horizon one of: intraday, days, weeks, months, unclear

You must not use any knowledge of what happened to this stock after these
posts were written. Judge only what the text itself says. Do not name any
ticker that does not appear in the text.
"""

FUNCTION llm_context(symbol s, day d, texts)
  IF NOT LLM_ENABLED: RETURN null

  key = (s, d)
  cache = LOAD LLM_CACHE
  IF key IN cache: RETURN cache[key] # one call per ticker per day, ever

  payload = {model: LLM_MODEL,
             temperature: LLM_TEMP,
             max_tokens: LLM_MAXTOK,
             messages: [{role: "system", content: SYS_PROMPT},
                       {role: "user", content: join(texts, limit=8000)}]}
  r = POST EP_LLM payload
  headers {Authorization: "Bearer " + LLM_KEY,
           Content-Type: "application/json"}
  TRY:
    out = parse_json(r.choices[0].message.content)
  EXCEPT:
    log_warning("LLM fail", s); RETURN null # never abort the pipeline

  cache[key] = out; SAVE LLM_CACHE
  RETURN out

# ---- STEP 3. DAILY AGGREGATION -----

```

```

FUNCTION aggregate_social(symbol s, day d)
# d = t-1 strictly. Posts from day t are not reliably available at close t.
docs = classified documents from calendar day d that mention s

# counted, free, applies to ALL symbols in the universe
soc_level      = count(docs)
soc_authors    = count(distinct author_hash IN docs)
soc_accel      = soc_level[d] / mean(soc_level[d-20:d]) - 1
concentration  = soc_level / max(soc_authors, 1)  # many posts, few authors
st_bull_ratio  = free_sentiment(s, d).st_bull_ratio

# model derived, applies to TOP-N ONLY, and only if LLM_ENABLED
ctx = llm_context(s, d, texts = top_20_texts_by_score(docs))
llm_conviction = ctx.conviction IF ctx ELSE null
llm_catalyst   = 1 IF ctx AND ctx.catalyst != null ELSE 0
llm_coordination = 1 IF ctx AND ctx.coordination ELSE 0
llm_novelty    = 1 IF ctx AND ctx.novelty ELSE 0
RETURN all of the above

# ---- COST, recomputed for THIS architecture -----
# calls = N_UNIVERSE names x 1 call per day x trading days
#       = 10 x 1 x approx 1,500 days = approx 15,000 calls per full pass
# tokens per call: approx 2,000 input (20 posts), 250 output
#       = approx 30M input, 3.75M output per pass
# deepseek/deepseek-v4-flash via OpenRouter: single-digit USD per pass.
#
# For contrast, the rejected design classified every individual document:
# 40,000,000 documents, filtered to 6..8M containing a ticker,
# 250..320M tokens, roughly USD 60 on a cheap model and USD 800..1,000
# on a frontier model, PER PASS, and each prompt revision needs a re-run.
# Overlaying the top-N is roughly two orders of magnitude cheaper and is the
# correct sequencing: prove the deterministic core first, add the model last.
#
# ---- LLM CADENCE. Daily, after the close. NEVER intraday. -----
# The model output feeds Layer 1, which runs once per day before the open.
# Layer 2 and the intraday confirmation are deterministic and must stay so.
# A model call inside a 15 minute loop adds latency and non-determinism to
# the only part of the system that has a measured edge.
# Schedule: 16:15 ET, after the close, before the next Layer 1 build.
#
# ---- MEASURED SIGNS. Feed these to gate_5 before assigning any weight. ----
# dd_from_ath      IC = +0.165  strongest overall. LESS fallen is better.
# prior_memes      IC = -0.152  names that already ran keep falling
# otm_call_level   IC = +0.061  strongest positive found
# soc_level        IC = -0.054  NEGATIVE. Already crowded means late.
# soc_accel        IC = +0.025  weakly positive
# dollar_vol_surge IC = +0.022  benchmark
# dtc, si_change   IC ~ 0       no signal at biweekly frequency
#
# WARNING. Every coefficient above was measured on the 195 name pool, which is
# 42% known meme names against a true base rate near 1%. They are observations,
# not established properties, and all must be recomputed on the neutral
# universe. The four llm_* features have NEVER been measured. Start W_LLM at 0.
END MODULE

```

6 Module 5. Layer 0, tradability

```

MODULE layer0

FUNCTION tradable(symbol s, day t)
f = features[s, t]
RETURN f.adv20_usd >= LIQ_FLOOR
      AND f.adv20_usd <= LIQ_CEIL
      AND f.price_raw >= PRICE_MIN

# APPLY THIS IDENTICALLY TO THE STRATEGY, THE RANDOM FLOOR, AND THE CEILING.
# DEFECT FOUND: the hindsight run had NO liquidity filter while the
# point-in-time run did. The comparison was meaningless and overstated the
# hindsight advantage by roughly $177,000. 669 of the missed trades were
# excluded by the filter, only 41 were genuinely outside the universe.
#
# CAPACITY CEILING, measured, and NOT fixable with more data:
# median 20-day dollar volume before a move of +100% is $861,000
# adv >= $1M -> lift 0.54x      adv >= $5M -> lift 0.35x
# The names that explode and the names that absorb capital are nearly disjoint.
# Decide the intended capital base BEFORE buying more data. At $10k per
# position the strategy is viable. At $100k it may not be.
END MODULE

```

7 Module 6. Layer 1, universe

```

MODULE layer1      # runs once per day, before the open, on data through t-1

FUNCTION build_universe(day t, weights W, thresholds TH)
  scored = []
  FOR s IN eligible_at(t):      # Module 22. NOT fetch_symbol_master.
                                # eligible_at already applies
                                # listed_on, has_bar and tradable.
    f = build_features(s, t)    # Module 3
    IF NOT gate_2_corporate_actions(f): CONTINUE

    # CROSS-SECTIONAL PERCENTILE, not an absolute threshold.
    # rank_pct(x, t) ranks x across every tradable symbol on day t only.
    # It looks at no other day, so it is point-in-time safe, and it
    # self-normalises to the volatility regime. See the measurement below
    # for why absolute thresholds were abandoned.
    IF rank_pct(f.atr_pct, t)    >= TH.ATR_Q      # e.g. top 10 percent
    AND rank_pct(f.ret_20d, t)   >= TH.RET20_Q     # e.g. top 10 percent
    AND rank_pct(-f.dd_from_ath,t) >= 0.50        # more fallen than median
    AND f.close                  > f.ma20:
      # z() is CROSS-SECTIONAL, computed across symbols on THIS day only.
      # It never looks forward and is always safe.
      # score_with_degradation (Module 17) drops any term whose input
      # is null and renormalises over the survivors. It returns null
      # when too little survives, and that name is simply not ranked.
      terms = [(W.ATR, f.atr_pct), (W.DD, -f.dd_from_ath),
                (W.VB, f.vol_build), (W.SOC, f.soc_accel),
                (W.LVL, -f.soc_level), (W.LLM, f.llm_conviction),
                (W.OPT, f.otm_call_level)]
      score, status = score_with_degradation(terms)
      IF status == "ok": scored.APPEND((score, s))
  RETURN top_n(SORT(scored, descending), TH.N_UNIVERSE)

# ---- MEASURED BASE RATES -----
# Denominator: 8,967,916 symbol-days. Event: a gain of +100% or more within
# the next 20 sessions. Unconditional base rate 0.68%.
#
# filter                hit rate    lift
# no filter              0.68%      1.00x
# adv20 >= $1M           0.37%      0.54x    <-- NEGATIVE
# adv20 >= $5M           0.24%      0.35x    <-- NEGATIVE
# above ma20 alone       0.58%      0.85x
# price_raw in [1, 20]   1.09%      1.59x
# vol_build >= 1.3        1.65%      2.40x
# ret_20d > +20%          1.94%      2.84x
# atr_pct >= 0.08         3.51%      5.12x
# dd_from_ath <= -0.90    3.78%      5.53x
# atr_pct >= 0.12         5.52%      8.05x
# COMBINED: adv>=1e6, atr>=0.12, above ma20,
#   ret20>0.20, dd<=-0.90      8.35%    12.19x
#   -> 19,184 symbol-days, about 7.2 names per day
#
# ---- WHY PERCENTILES REPLACED ABSOLUTE THRESHOLDS -----
# An absolute gate produces a universe whose SIZE depends on the regime.
# Measured, names per day by year, 2021 through 2026:
#
# gate                  avg/day  2021..2026      CV    lift
# ABSOLUTE atr>=12%, ret20>20%,
#   dd<=-90%, >MA20       7.2  [3 3 8 16 23 23]  0.74  12.19x
# PERCENTILE top 20% + dd>median  92.8 [89 81 125 137 136 108] 0.21 4.02x
# PERCENTILE top 15% + dd>median  65.6 [62 52 90 100 94 76]  0.24 4.98x
# PERCENTILE top 10% + dd>median  40.5 [37 30 57 65 56 46]  0.27 6.31x
# PERCENTILE top 15% + gapfreq>=25% 30.5 [36 25 30 47 69 58]  0.39 8.70x
#
# CV is the coefficient of variation of the daily universe size.
#
# The absolute gate has the highest lift and is UNUSABLE. It yields 3 names per
# day in 2021 and 23 in 2025, an eightfold drift. With MAX_POS = 10 the 2021
# and 2022 folds cannot even fill the position budget, so those folds would
# measure how many names passed the filter, not how well the system selects.
# A walk-forward run on it compares universe SIZE across folds, not SKILL.
#
# RECOMMENDED: percentile top 10 percent with drawdown above the median.
# 40 names per day, roughly four times the position budget, CV 0.27, and a
# lift of 6.31x which is still far above anything else measured in this study.
#
# ---- ADDITIONAL GATES DO NOT HELP. MEASURED. -----
# Every candidate below is point-in-time and unbiased. Standalone lift on the
# full universe, then incremental lift when ADDED to the base gate:
#
# gate                  standalone  added to base  names/day

```

```

# gap frequency >= 25% over 20d      5.19x      0.99x      6.3
# dollar volume accel adv5/adv60>=2  2.90x      1.08x      2.6
# ATR compression atr20/atr60<=0.8  2.24x      --      --
# ATR expansion atr20/atr60>=1.3      2.17x      0.95x      5.2
# distance from 52w low >= +50%      1.31x      1.01x      6.7
# listing age <= 500 sessions          1.21x      1.33x      0.8
# up-day ratio over 20d >= 0.6        0.55x      1.11x      1.6
# position in 20d range >= 0.8        0.51x      --      --
#
# Gap frequency is the strongest NEW signal found, at 5.19x standalone, and it
# had never been tested. Added to the base gate it contributes 0.99x, meaning
# nothing. A stock with ATR above 12 percent already gaps frequently; the two
# select the same names. Every other candidate either adds under 1.1x or cuts
# the universe below the position budget. The conclusion is that the base gate
# has already extracted the available cross-sectional signal from price and
# volume, and more price-and-volume gates are redundant.
#
# ---- THE SQUEEZE THESIS DOES NOT SURVIVE MEASUREMENT -----
# days_to_cover < 2 lift 1.44x
# days_to_cover >= 3 lift 0.60x <-- INVERTED vs the thesis
# si_change median before a large move: 0%
# Both earlier screens ranked on days to cover and both failed for this reason.
# Screen v1 ranked on dtc and selected Canadian banks and pipelines (IMO, FTS,
# TRP, RY, CM) whose dtc is inflated because US volume in the denominator is
# thin. Keep short interest as a DIAGNOSTIC, not a ranking weight, until borrow
# fee data exists to test the mechanism directly.
END MODULE

```

8 Module 7. Layer 2, ignition trigger

```

MODULE layer2      # evaluated at the CLOSE of day t

FUNCTION scan_triggers(universe, day t)
  queue = []
  FOR s IN universe:
    f = build_features(s, t)      # same feature table as Module 6
    typical = (high[t] + low[t] + close[t]) / 3
    IF f.rvol >= RVOL_MIN      # unusual participation
    AND f.close > typical      # closed in the upper part of range
    AND f.close > close[t-1]   # rejects a one day pop
    AND f.close > f.ma20       # trend filter
    AND f.ret_5d > MOM5_MIN     # rejects sideways drift
    AND f.pct_vs_ma20 <= EXT_CAP # not already exhausted
    THEN queue.APPEND((f.rvol, s))
  RETURN SORT(queue, descending)

# TIMING, measured: on days that BEGIN a move of +100% or more, median rvol is
# 0.97x, meaning volume is NORMAL. Volume rises AFTER the move starts.
# What DOES precede these moves is vol_build at median 1.28x, a slow month-long
# accumulation. A trigger keyed to a volume spike is therefore structurally
# late, and the accumulation term belongs in Layer 1, not here.

MODULE layer2b     # optional intraday confirmation, next session

FUNCTION confirm_intraday(s)
  # Only symbols that passed layer2 are queried, so this covers ~10 symbols
  # rather than 5,000. That is the whole point of the two layer split.
  FOR bar IN 15min_bars(s, session t+1):
    cum_vol = sum(volume from open to this bar)
    base = mean(cum_vol at the SAME time-of-day slot, prior 20 days)
    slot_rvol = cum_vol / base
    # time-of-day adjustment is required. Comparing 10:00 volume to a
    # full-day average flags every stock every morning.
    IF slot_rvol >= 2.0 AND price > vwap: RETURN "ENTER" AT bar.close
    IF bar.time >= 15:30: RETURN "STAND_DOWN"
END MODULE

```

9 Module 8. Execution and position management

```

MODULE execution

FUNCTION place_entries(queue, day t, equity)
  FOR (score, s) IN queue:
    IF count(open_positions) >= MAX_POS: BREAK

```

```

IF gate_6_solvency(equity) == "HALT_NEW_ENTRIES": BREAK
ok, why = admit_position(s, open_positions, t) # Module 15
IF NOT ok: log(s, "rejected:" + why); CONTINUE
f = build_features(s, t)
entry = open[t+1] # NEVER close[t]
IF entry IS null OR entry <= 0: CONTINUE
cost = 2 * min(0.03, 0.0005
              + f.atr_pct * sqrt(POS_SIZE / f.adv20_usd))
open_position(s, entry, POS_SIZE, cost)

# DEFECT FOUND: deciding and filling at the same closing price overstated
# expectancy by 38%, from +2.77% to +4.48%.

FUNCTION manage_positions(day j)
  FOR p IN open_positions:
    check_position_events(p, j) # Module 16. May exit and return.
    IF p IS closed: CONTINUE
    IF halted(p.symbol, j): # Module 14
      handle_halt(p, halt_event(p.symbol, j)); CONTINUE
    f = build_features(p.symbol, j)
    stop = p.entry * (1 - HARD_STOP)
    # order matters. Gap check must precede the intraday touch check.
    IF open[j] <= stop: close_at(open[j], "stop_gap")
    ELIF low[j] <= stop: close_at(stop, "stop")
    ELIF age(p) >= EARLY_DAYS
      AND mfe(p) < EARLY_MFE: close_at(close[j], "no_progress")
    ELIF close[j] <= peak(p) - ATR_MULT * f.atr14:
      close_at(close[j], "trail")
    ELIF close[j] > p.entry
      AND close[j] < ma(close, MA_EXIT): close_at(close[j], "trend_break")
    ELIF age(p) >= MAX_HOLD: close_at(close[j], "max_hold")

# ---- MEASURED EXIT BEHAVIOUR, 1,167 trades -----
# trend break      323 trades  win 100.0%  avg +30.4%  +$981,781
# trailing stop    20 trades  win 60.0%  avg +41.0%  +$81,975
# max hold         5 trades  win 40.0%  avg +0.6%   +$275
# no progress      419 trades win 20.0%  avg -2.6%  -$109,988
# hard stop        400 trades win  0.0%  avg -11.8% -$473,368
#
# All 10 BEST trades exited on trend break. All 10 WORST exited on hard stop,
# 7 of them within 2 sessions, realised losses 22.8% to 39.1% against a nominal
# 8% stop. That difference is overnight gap risk. No stop placement prevents
# it. The only defence is position size.
#
# ---- ON THE WIN RATE, since it will be asked -----
# Every filter that RAISES win rate LOWERS total profit:
# price >= $5      win 37.2%  expectancy +2.77%
# adv >= $50M      win 37.8%  expectancy +3.33%
# no filter        win 36.1%  expectancy +4.12%
# rvol >= 5        win 32.7%  expectancy +9.32%
# Average winner +30.4%, average loser -11.8%, ratio 2.6 to 1, so break-even
# sits at a 28% win rate. At 36% there is margin. Win rate is the wrong
# objective for this payoff shape.
#
# ---- ON EXIT TIMING -----
# Median retracement from the peak within 60 sessions is -55% for moves above
# +100%, and -67% for moves above +200%. Holding through is not viable.
#
# ---- HISTORICAL NOTE. DO NOT REINSTATE THE OLD RULE. -----
# An earlier version closed any position at or below entry after 3 sessions.
# It GUARANTEED a loss on 39% of trades and cost $40,000. It was replaced by
# the no-progress exit above (3% MFE after 2 days), which gained $21,000 and
# reduced winners above +50% only from 37 to 35.
END MODULE

```

10 Module 9. Walk-forward driver

This module is the experiment. Everything above is mechanics.

```

MODULE walkforward

# ---- WHY THE PREVIOUS VALIDATION FAILED -----
# 1. An earlier walk-forward split into three folds of EQUAL TRADE COUNT and
# all three were positive. Splitting by trade count rather than by calendar
# date concealed that the two most recent years were negative.
# 2. A null test against random ENTRY TIMING passed at p = 0.025 while the
# universe was invalid, because it held the universe fixed. A control must
# randomise the same thing the strategy chooses. The strategy chooses NAMES,
# so the control must choose names randomly.

```

```

CONST WARMUP_START = 2016-01-01 # establishes all-time highs only. No trades.
CONST SAMPLE_START = 2021-01-01 # first date where SI and social both exist
CONST TRAIN_MONTHS = 24
CONST TEST_MONTHS = 6
CONST STEP_MONTHS = 6
CONST PURGE_MONTHS = 1

# The purge gap is REQUIRED. The label uses a forward 20 session window and
# MAX_HOLD is 60 sessions. Without it a trade opened at the end of training is
# still open during testing and the two windows are not independent.
#
# fold 1 train 2021-01..2022-12 purge 2023-01 test 2023-02..2023-07
# fold 2 train 2021-07..2023-06 purge 2023-07 test 2023-08..2024-01
# fold 3 train 2022-01..2023-12 purge 2024-01 test 2024-02..2024-07
# fold 4 train 2022-07..2024-06 purge 2024-07 test 2024-08..2025-01
# fold 5 train 2023-01..2024-12 purge 2025-01 test 2025-02..2025-07
# fold 6 train 2023-07..2025-06 purge 2025-07 test 2025-08..2026-01
# fold 7 train 2024-01..2025-12 purge 2026-01 test 2026-02..2026-07

FUNCTION run_walkforward()
  results = []
  FOR fold IN folds:

    # ----- IN SAMPLE: fit -----
    # Only these may be fitted. Everything in Module 0 marked CONST is
    # fixed for the whole study and is NOT touched here.
    best = null
    FOR W, TH IN parameter_grid(W_ATR, W_DD, W_VB, W_SOC, W_LVL, W_LLM,
                                W_OPT, N_UNIVERSE, ATR_FLOOR, DD_FLOOR,
                                RET20_FLOOR):
      ASSERT all data touched has date <= fold.train_end
      m = simulate(fold.train_start, fold.train_end, W, TH)
      IF m.sharpe > best.sharpe: best = (W, TH)

    # ----- LEAKAGE ASSERTIONS -----
    ASSERT no z-score, percentile, or normalisation constant was computed
      across the full sample. All are per-day cross-sectional or
      trailing-window only.
    ASSERT for every test day t, universe(t) used data through t-1 only
    ASSERT every FINRA row used satisfies settle + SI_LAG_DAYS <= t
    ASSERT every social document used has created_utc < t
    ASSERT no position opened during train is still open during test
    ASSERT price LEVEL filters read px_raw, and returns read px_split
    ASSERT symbol list was never modified by hand

    # ----- OUT OF SAMPLE: evaluate once -----
    strat = simulate(fold.test_start, fold.test_end, best.W, best.TH)
    floor = simulate_random(fold.test_start, fold.test_end,
                           n_per_day = best.TH.N_UNIVERSE)
    ceil = simulate_biased(fold.test_start, fold.test_end) # scale only
    bench = buy_and_hold("SPY", fold.test_start, fold.test_end)
    # all four carry identical costs, identical tradability, identical exits

    results.APPEND({fold, strat, floor, ceil, bench, chosen: best})
  RETURN results

# --- CONTROLS ---
# STRATEGY the pipeline above
# FLOOR N_UNIVERSE symbols drawn at random each day from the same
# tradability-filtered pool. Same trigger, same exits, same costs.
# CEILING a deliberately biased universe of names known to have run.
# For SCALE ONLY. Never reported as an achievable result.
# BENCHMARK SPY buy and hold over the identical window.
#
# --- MEASURED REFERENCE VALUES, same window ---
# SPY buy and hold CAGR +15.9% Sharpe 0.95 maxDD -25.4%
# biased 82 names CAGR +32.7% Sharpe 1.07 maxDD -19.4% $580,674
# screen v1, valid CAGR +12.9% Sharpe 0.61 maxDD -36.3% $212,817
# random control CAGR +3.4% Sharpe 0.26 $123,401
# The only admissible version so far sits at 0.61, BELOW SPY at 0.95.
# 0.95 is the number to beat, not zero.

# --- ACCEPTANCE CRITERIA. Fix these BEFORE running, not after. ---
FUNCTION accept(results)
  RETURN aggregate_sharpe(results.strat) > 0.95 # must beat SPY
  AND count(folds with expectancy > 0) >= 5 of 7
  AND aggregate_expectancy(strat) - aggregate_expectancy(floor)
    > 2 * standard_error
  AND aggregate_profit(strat, excluding top 10 trades) > 0
  AND beta_to_spy(strat) < 0.30
  AND median_cost_per_round_trip < gross_expectancy / 3

```

```

# Criterion 4 answers the concentration objection directly. The thesis DOES
# depend on large winners and that is legitimate. But a strategy that is
# negative without its ten best trades has no measurable base and cannot be
# separated from luck at this sample size. In the biased run the top 10 trades
# were 71% of all profit.
END MODULE

```

11 Module 10. Evaluation

```

MODULE evaluate

FUNCTION report(trades, daily_equity, benchmark)
  # PER TRADE
  n, win_rate, expectancy, mean_cost
  exit_reason_breakdown
  top_10 and bottom_10 with entry date, entry, exit date, exit, reason
  profit_share_of_top_10

  # RISK, computed on the DAILY MARK TO MARKET curve
  r = daily_equity.pct_change()
  sharpe = mean(r) / std(r) * sqrt(252)
  sortino = mean(r) / std(r WHERE r < 0) * sqrt(252)
  calmar = cagr / abs(max_drawdown)
  max_drawdown, AND dollar profit reported beside it

  # A per-trade mean divided by a per-trade standard deviation is NOT a
  # Sharpe ratio. It has no time axis. Cross-check:
  #   per_trade_sharpe * sqrt(trades_per_year) ~= annualised sharpe
  #   measured once: 0.08 * sqrt(105) = 0.82 vs 0.84 computed properly

  # VERSUS BENCHMARK
  beta, alpha_annualised, correlation vs SPY
  # measured on the biased run: beta 0.149, alpha +30.5%, correlation 0.082.
  # Low correlation is the one genuinely valuable property found so far.

FUNCTION report_ic(features, universe = NEUTRAL)
  FOR f IN features:
    FOR h IN {5, 10, 20}:
      daily_ic = spearman(f[all symbols, day d],
                          forward_return[all symbols, d : d+h])
      ic[f, h] = mean(daily_ic over all days)
  RETURN ic

# PREFER THE INFORMATION COEFFICIENT WHEN THE TWO DISAGREE.
# A backtest of 300 trades with a heavy right tail cannot separate hypotheses.
# In one window a RANDOM control returned +8.02% per trade and ranked first
# among all variants. 30,000 cross-sectional observations can separate them.
END MODULE

```

12 Module 11. Missing data, and the gap each one closes

```

MODULE missing_data

# ===== 11.1 FIXES A KNOWN ERROR IN NUMBERS ALREADY REPORTED =====

# RAW UNADJUSTED PRICES
# gap price level filters currently read split-adjusted prices, which
# embed future corporate actions. 7.8% of events affected.
# source Alpaca, adjustment=raw. Already paid for.
# cost free, about 30 minutes of fetching
# fixes an ACTIVE look-ahead leak. Cheapest item on this list. Do it first.

# DELISTED SECURITIES
# gap symbol master holds active listings only. 27 of the original 82
# names, 33%, are already absent. Every PIT result is optimistic by
# an unmeasured amount. Matters far more for a long-only strategy.
# source SOLVED AND FREE. See Module 22. FINRA biweekly files provide
# point-in-time listing membership, and the Alpaca data endpoint
# serves bars for delisted symbols even when the assets endpoint
# has purged them. Polygon and Norgate are NOT required.
# cost free. Roughly one hour of additional fetching.
# fixes survivorship bias for 2021 onward. Pre-2021 remains unsolved.

# SHARES OUTSTANDING AND FREE FLOAT
# gap market capitalisation is entirely absent. Short interest as a

```

```

# percentage of float is discussed with no denominator.
# source GET_EP_EDGAR/CIK{10-digit}/us-gaap/CommonStockSharesOutstanding.json
# cost free, full history
# fixes enables a real size filter and a real float-based short measure

# ===== 11.2 TESTS AN OPEN QUESTION =====

# SOCIAL TEXT ACROSS THE NEUTRAL UNIVERSE
# gap collected for 195 names only, and those 195 are 42% known meme
# names. Every social coefficient is measured on a contaminated
# sample and may not generalise.
# source Arctic Shift, free, one overnight run per pass.
# LLM overlay via OpenRouter deepseek-v4-flash, cached per
# (ticker, day), single-digit USD per full pass.
# tests whether attention acceleration selects names BEFORE a move

# SOCIAL DEPTH: AUTHOR IDENTITY AND VOTES
# gap only counts were stored, so coordination cannot be separated from
# one account posting repeatedly
# tests whether coordination, as distinct from volume, is the real signal.
# This is precisely what a mention count cannot express and is the
# main reason to involve a language model at all.

# OPTIONS ACROSS THE NEUTRAL UNIVERSE
# gap otm_call_level has the strongest positive IC found, +0.061, but was
# measured on 110 names from the biased pool and only from 2024
# source Alpaca OPRA, already accessible
# tests whether the only surviving positive signal is real

# BORROW FEE AND UTILISATION
# gap NEVER measured. No coefficient exists because no history exists.
# Every claim about its importance rests on squeeze mechanics, not on
# evidence produced here. Given that days to cover turned out to be
# INVERTED, this assumption deserves scepticism.
# source Ortex or Fintel, paid; or start a forward daily record today
# tests whether borrow pressure is the mechanism at all

# ===== 11.3 SUPPORTING =====
# DILUTION FILINGS, S-3 AND ATM SEC EDGAR full text search, free.
# tests whether dilution explains the median -55% retracement after the peak
# FAILS TO DELIVER SEC, free, 2004 onward, verified retrievable

# ===== 11.4 WHAT NO DATA CAN FIX =====
# Median 20-day dollar volume before a +100% move is $861,000. Liquidity
# filters reduce the hit rate monotonically: 0.54x at $1M, 0.35x at $5M.
# The names that explode and the names that absorb capital are nearly disjoint.
# This is a CAPACITY CEILING, not a data gap. Establish the intended capital
# base before spending further on data.
END MODULE

```

13 Module 12. Failure modes to assert against

```

MODULE failure_modes

# 1. Any filter tuned until it reproduces a list of known winners. Circular.
# Coverage of a known list is NOT a success metric. Treating it as one is
# how the original bias entered. Measured consequence: adding a volatility
# floor raised coverage of the 82 names from 24 to 46 of 54 and moved
# expectancy from +0.67% to -1.66%. The better a screen reproduced the
# known list, the worse it performed.
# 2. A robustness test run inside a biased universe. Four of six such tests
# passed on a universe later shown to be invalid.
# 3. Percentile thresholds computed over the full sample. Use trailing windows.
# 4. A feature whose sign was chosen by intuition. Measure first. See gate_5.
# 5. A control that randomises something other than what the strategy chooses.
# 6. A Sharpe ratio computed across trades with no time axis.
# 7. Any prompt that invites a model to RECALL the past rather than READ it.
END MODULE

```

14 Module 13. Trending detection

```

MODULE trending

# ApeWisdom is the only free source that gives a RANK, not just a count.
# A rank CHANGE is more useful than a level. The level says a name is already

```

```

# crowded, and its measured IC is -0.054, NEGATIVE. The rank change says
# attention is arriving, which is the thing the strategy actually wants.

FUNCTION fetch_apewisdom(day d)
    rows = []
    FOR page IN 1..5:
        r = GET "https://apewisdom.io/api/v1.0/filter/wallstreetbets/page/" + page
        rows.APPEND(r.results) # {ticker, mentions, mentions_24h_ago, rank}
    RETURN rows
# NO HISTORY. This returns TODAY only and cannot be backtested.
# Backtest uses Arctic Shift, live uses this. They are DIFFERENT SERIES.
# See Module 20, which exists entirely because of this asymmetry.

FUNCTION define_trending(symbol s, day d)
    # three competing definitions. Measure each separately via gate_5.
    # Do NOT blend them before measuring.
    lvl = mentions[s, d]
    accel = mentions[s, d] / mean(mentions[s, d-20:d]) - 1
    rank_delta = rank[s, d-1] - rank[s, d] # positive = climbing

    is_trending_level = lvl >= pct(all_mentions[d], 90) # cross-sectional
    is_trending_accel = accel >= 2.0 # 3x the 20 day mean
    is_trending_rank = rank_delta >= 20 AND rank[s, d] <= 50

    # STATUS: soc_level IC -0.054, soc_accel IC +0.025, rank_delta NEVER
    # measured. None may enter the composite before gate_5 clears it.
    RETURN {is_trending_level, is_trending_accel, is_trending_rank,
            lvl, accel, rank_delta}

FUNCTION define_no_chance(symbol s, day d)
    # the REVERSE screen. Drop these first, cheaply, before any scoring.
    RETURN adv20_usd > 500000000 # too large to move on retail flow
        OR atr_pct < 0.02 # structurally quiet
        OR mentions[s, d-60:d] all zero # no community exists at all
END MODULE

```

15 Module 14. Halts and limit up limit down

```

MODULE halts

# THIS IS WHY realised stop losses were -11.8% against a nominal -8%, and why
# the worst reached -39.1%. It was the largest omission in the earlier spec.
#
# LULD bands, NMS Plan:
# Tier 1 (S&P 500, Russell 1000, select ETPs) band 5%
# Tier 2 (everything else, so every meme name) band 10%
# price in [0.75, 3.00] band 20%
# price below 0.75 band = min(75%, $0.15 / price)
# DOUBLED during 09:30-09:45 and 15:35-16:00
#
# A typical meme name at $4 carries a 10% band, 20% at the open. A stop at 8%
# therefore sits INSIDE the band. Price passes through it with no halt at all.

FUNCTION luld_band(price, tier, time_of_day)
    IF tier == 1: base = 0.05
    ELIF price >= 3.00: base = 0.10
    ELIF price >= 0.75: base = 0.20
    ELSE: base = min(0.75, 0.15 / price)
    IF time_of_day IN [09:30-09:45] OR [15:35-16:00]: base = base * 2
    RETURN base

FUNCTION handle_halt(position p, halt_event h)
    IF h.type == "LULD_PAUSE": # minimum 5 minutes, often longer
        mark_position_unfillable(p) # a stop is NOT protection during a halt
        ON reopen: fill at the AUCTION PRINT, not at the stop level
    IF h.type IN {"T1", "T12"}: # news pending, SEC suspension
        mark_position_frozen(p) # T12 frequently precedes delisting
        value at last print, flag for manual review

FUNCTION probable_halt_day(bar, prev_close, tier)
    # Daily bars do not record halts. Infer them.
    RETURN (bar.high - bar.low) / prev_close
        > 2 * luld_band(prev_close, tier, "mid")
    # On a flagged day do NOT assume a clean fill at the stop price. Assume
    # the worse of {open, stop}. Module 8 already does this, which is why its
    # gap branch MUST be evaluated before its low-touch branch.
END MODULE

```

16 Module 15. Concentration and correlation limits

```
MODULE concentration

# MAX_POS = 10 does not bound risk if all 10 are the same trade.
# Observed repeatedly in the measured runs: crypto miners MARA, RIOT, BTBT,
# SDIG and CORZ all ignite on one Bitcoin move. Quantum names RGTI, QUBT,
# IONQ and LAES all ignite on one news cycle.
# Ten positions in one theme is one position at ten times the size.

FUNCTION theme_of(symbol s, day t)
    # No free sector feed covers microcaps well, and a static sector label
    # would miss the crypto and quantum clusters anyway. Derive it from
    # co-movement, computed through t-1 only.
    RETURN cluster_id from correlation_clustering(returns[t-120:t], k = 20)

FUNCTION admit_position(candidate s, open_positions, day t)
    IF count(p IN open_positions WHERE theme_of(p, t) == theme_of(s, t))
        >= MAX_PER_THEME:
        RETURN false, "theme_cap"
    FOR p IN open_positions:
        IF corr(returns[s, t-60:t], returns[p.symbol, t-60:t])
            > MAX_CORR_PAIRWISE:
            RETURN false, "correlated_with_" + p.symbol
    RETURN true, "ok"

# REPORT the realised maximum theme concentration per fold. If the strategy
# only works when allowed to put all 10 slots into one theme, then the
# reported drawdown understates the true risk and the result is not usable.
END MODULE
```

17 Module 16. Corporate actions while holding

```
MODULE corporate_actions

FUNCTION check_position_events(position p, day j)
    # These names reverse-split, dilute and delist far more often than the
    # broad market. 27 of the original 82 names are already gone.

    IF split_occurred(p.symbol, j):
        ratio = split_ratio(p.symbol, j)
        p.entry = p.entry / ratio          # adjust the record, not the P&L
        p.shares = p.shares * ratio
        p.stop = p.stop / ratio
        IF ratio < 1: flag(p, "reverse_split_while_held")
        # a reverse split on a name you hold is a warning, not a neutral
        # event. It is usually done to preserve a listing requirement.

    IF delisting_notice(p.symbol, j):
        exit_immediately(p, "delisting")    # do not wait for a trigger

    IF offering_filed(p.symbol, j):         # S-1, S-3, 424B5, ATM program
        exit_immediately(p, "dilution_filed")
        # SOURCE: https://efits.sec.gov/LATEST/search-index?q=%22{ticker}%22
        #           &forms=424B5,S-3
        # Dilution is the most common way a squeeze ends, and median
        # retracement from peak is -55%. But this rule is a HYPOTHESIS.
        # Backtest it as an ablation before enabling it. NOT YET MEASURED.

    IF ticker_reused_after_bankruptcy(p.symbol, j):
        FAIL "price series discontinuity on a held name"
        # this produced GPOR +52,648% and AQB +367,817% in the raw event
        # table. Gate 2 removes them from analysis. Here it must also
        # invalidate any live position.
END MODULE
```

18 Module 17. Graceful degradation

```
MODULE degradation

# Every external source WILL fail. Measured failures in this project:
#   reddit.com public JSON          permanent 403
#   api.stocktwits.com             intermittent 403 and 429
```

```

# iborrowdesk.com          403 until a browser User-Agent was sent
# yfinance                 429 across all 120 tickers at once
# Arctic Shift             422 and 429 under load
# Alpaca free tier         403 on recent SIP data
#
# A pipeline that ABORTS on any one of these never finishes an overnight run.
# A pipeline that SILENTLY SUBSTITUTES ZERO produces a false conclusion, which
# is exactly what happened when the social features all became zero.
# The only safe behaviour is: degrade, record, and REFUSE TO SCORE.

CONST TIER = {
    price_bars: "REQUIRED",    # no substitute exists, abort the day
    symbol_master: "REQUIRED",
    short_interest: "OPTIONAL", # IC ~ 0, weight already near zero
    social: "OPTIONAL",
    options: "OPTIONAL",
    llm: "OPTIONAL",          # LLM_ENABLED defaults false in any case
}

FUNCTION safe_fetch(source, fetch_fn, day d)
    TRY:
        v = fetch_fn(d)
        health[source, d] = "OK"
        RETURN v
    EXCEPT timeout, 4xx, 5xx AS e:
        health[source, d] = "FAIL:" + e
        IF TIER[source] == "REQUIRED":
            HALT "required source unavailable, no trading decision made"
        RETURN null          # null. NOT zero. null is not zero.

FUNCTION score_with_degradation(symbol s, day t)
    terms = []; weight_used = 0
    FOR (feature, weight) IN composite_terms:
        v = features[s, t][feature]
        IF v IS null: CONTINUE      # drop the TERM, never zero it
        terms.APPEND(weight * z(v)); weight_used += abs(weight)
    IF weight_used < 0.60 * total_weight:
        RETURN null, "insufficient_data" # refuse to rank this name today
    RETURN sum(terms) / weight_used, "ok" # renormalise over what survived

# Zeroing a missing z-score asserts the name is exactly average on that
# feature. That is a CLAIM, not an absence. Renormalising over the surviving
# terms makes no claim, which is the correct behaviour under missing data.

FUNCTION daily_health_report()
    PRINT per source: rows fetched, coverage vs expected, first and last date
    PRINT symbols scored, symbols refused for insufficient_data
    ASSERT coverage > 0.5 * expected FOR every REQUIRED source
    # gate_1 promoted from a one-off check to a daily operational gate
END MODULE

```

19 Module 18. Operations

```

MODULE operations

# ---- SECRETS -----
# This project leaked keys into notebooks and had to gitignore nine files
# after the fact. The Alpaca AK key is a LIVE ACCOUNT key.
FUNCTION load_secrets()
    ASSERT no key appears in any .ipynb, .py, or .tex file
    ASSERT .gitignore covers meme_cache/, every notebook holding a key, .env
    ASSERT no code path in the research repo reaches POST /v2/orders
    # Every research script performs GET requests only. This is enforced by
    # assertion, not by intention.
    RETURN {k: env(k) FOR k IN [APCA_API_KEY_ID, APCA_API_SECRET_KEY,
                                DEEPSEEK_API_KEY]}

# ---- RESUMABILITY -----
# A full universe fetch is 18.5M bars and takes over an hour. A Reddit pass
# runs overnight. Neither may restart from zero.
FUNCTION resumable_fetch(work_items, checkpoint_file, fetch_fn)
    done = LOAD checkpoint_file IF exists ELSE {}
    FOR item IN work_items:
        IF item IN done: CONTINUE
        TRY:
            result = fetch_fn(item)
            SAVE result TO partial_store
            done[item] = timestamp
            IF count(done) MOD 5 == 0: SAVE done TO checkpoint_file

```

```

    EXCEPT rate_limit:
        sleep(exponential_backoff); retry up to 5
    SAVE done TO checkpoint_file
    RETURN consolidate(partial_store)

# ---- BACKUP -----
# every 30 minutes copy the cache directory, retain the last 10 versions.
# An overnight run that is lost costs a full day.

# ---- STORAGE SCHEMA -----
# bars_split.parquet    date, ticker, open, high, low, close, volume
# bars_raw.parquet      same, adjustment=raw, for PRICE LEVEL only
# finra_all.parquet     ticker, short_int, prev_si, adv, dtc, settle,
#                       usable_from
# social_docs.parquet   doc_id, created_utc, author_hash, score, body_text,
#                       tickers[]
# options_flow.parquet  date, ticker, otm_call_volume
# borrow_daily.parquet  date, ticker, fee_pct, available # forward record
# llm_context.json      {(ticker, date): {conviction, catalyst, ...}}
# health.parquet        date, source, status, rows, coverage
# Partition price data by year. 18.5M rows is 336MB unpartitioned.
END MODULE

```

20 Module 19. Orders and slippage in live trading

```

MODULE orders

# The backtest fills at open[t+1]. Live must reproduce that, or the backtest
# is not a forecast of the live system.

FUNCTION submit_entry(symbol s, day t_plus_1)
    # A market order into a thin opening auction is how a $10,000 order
    # becomes a 3% slippage event. Use a marketable limit and measure.
    ref = last_close
    limit = ref * (1 + ENTRY_LIMIT_PCT)
    order = {symbol: s, qty: floor(POS_SIZE / ref), side: "buy",
            type: "limit", limit_price: limit, time_in_force: "opg"}
    # if unfilled at the auction, DO NOT CHASE. Record a missed signal.
    # A backtest filling 100% of signals while live fills 80% is not the
    # same strategy, and the difference is not a rounding error.

FUNCTION submit_exit(position p, reason)
    IF reason IN {"stop", "stop_gap"}:
        type = "market" # a stop must fill. Never limit it.
    ELSE:
        type = "limit" AT last_price * 0.995

FUNCTION reconcile(day d)
    RECORD fill_price vs assumed open[t+1] -> entry_slippage
    RECORD fill_rate vs signals generated -> capture_rate
    RECORD realised cost vs modelled cost -> cost_model_error
    # the cost model 0.0005 + atr_pct * sqrt(POS_SIZE / adv20_usd), capped at
    # 3% one way, has NEVER been validated against a real fill.
    IF abs(cost_model_error) > 0.5 * modelled: FLAG "recalibrate cost model"
END MODULE

```

21 Module 20. Source parity and staged validation

```

MODULE parity

# THE ASYMMETRY PROBLEM. The backtest uses Arctic Shift, which has history but
# no live feed. Live uses ApeWisdom, which has a live feed but no history.
# These are DIFFERENT SIGNALS. A strategy validated on one and deployed on the
# other is untested.

FUNCTION validate_source_parity()
    FOR d IN forward_recording_period:
        arctic = arctic_shift_mentions(d)
        ape = apewisdom_mentions(d)
        RECORD spearman(arctic, ape) cross-sectionally
    ASSERT median_correlation > 0.7 BEFORE trusting the backtest for live use
    # if they disagree, the backtest measures a signal the live system will
    # never see, and the walk-forward result does not transfer.

# ---- STAGED ROLLOUT. Do not skip a stage. -----
# STAGE 1 walk-forward, Module 9. Must pass all six acceptance criteria.

```

```

# STAGE 2 paper trading, Alpaca paper endpoint, minimum 3 months.
# Each day, store the signals the LIVE scanner produced. Months
# later, run the BACKTEST over those same days. The two name lists
# must match symbol for symbol.
# STAGE 3 live at one tenth size, minimum 3 months, reconcile every fill.
# STAGE 4 full size, only if stage 3 cost and capture match the model.
#
# STAGE 2 IS THE ONLY TEST THAT CATCHES A LEAK THE ASSERTIONS MISS.
# If the backtest names a stock the live scanner did not, the backtest saw
# something the live system could not. Every assertion in Module 9 can pass
# and this test can still fail.
END MODULE

```

22 Module 21. Universe churn

```

MODULE churn

# The universe is rebuilt every day. A held name can leave it. The earlier
# spec did not say what happens then, which is an ambiguity a builder would
# have to resolve by guessing.

FUNCTION on_universe_rebuild(new_universe, open_positions)
  FOR p IN open_positions:
    IF p.symbol NOT IN new_universe:
      CONTINUE # DO NOTHING. Do not exit.
      # Universe membership gates ENTRY and never EXIT. Exits are
      # governed by Module 8 alone. Mixing the two would create an exit
      # rule that was never backtested, and it would fire on exactly
      # the names that are working: a stock whose price has doubled now
      # fails dd_from_ath and fails the exhaustion cap.
  # The ONLY exception is Module 16: delisting or a dilution filing.

FUNCTION universe_churn_report()
  turnover = |U(t) symmetric_difference U(t-1)| / |U(t)|
  PRINT daily turnover, and its 20 day mean
  ASSERT mean_turnover < MAX_CHURN
  # High churn means the score is noise driven and the watchlist is
  # unstable. Target below 30% daily. NOT YET MEASURED.
END MODULE

```

23 Module 22. Point-in-time listing membership

This module removes survivorship bias. It replaces `fetch_symbol_master` in Module 1.

```

MODULE pit_universe

# =====
# THE PROBLEM
# Scanning today's listed symbols and backtesting them over 2021 to 2026 asks
# "how would this strategy have done on the companies that survived". That is
# not the question. A trader in 2021 could buy BBBY, IRNT, SPRT and MULN. All
# four are gone. A long only strategy that never sees the names that went to
# zero reports a return no one could have earned.
#
# MEASURED SIZE OF THE GAP
# FINRA snapshot 2021-08-31 : 20,123 symbols
# FINRA snapshot 2026-07-31 : 22,341 symbols
# present in 2021, absent in 2026 : 8,437 symbols
# after filtering to plausible common stock tickers : 8,078 symbols
# Those 8,078 are invisible to a survivor-only scan.
#
# THE TWO FACTS THAT MAKE THIS FIXABLE FOR FREE
# 1. Every FINRA biweekly file is a POINT-IN-TIME SNAPSHOT of every NMS
# security that existed on that settlement date. Verified:
#   shrt20210831.csv contains BBBY, SPRT, IRNT, GME, AMC
#   shrt20220429.csv contains BBBY, MULN, BBIG, EXPR
#   shrt20230731.csv NO LONGER contains BBBY (delisted April 2023)
# The file set is therefore a free, dated membership record from 2021.
# 2. The Alpaca DATA endpoint still serves bars for delisted symbols even
# after the ASSETS endpoint has purged them. Verified: BBBY, IRNT, SPRT
# and MULN all return HTTP 200 with real bars. A batch request mixing
# live and dead symbols also returns 200.
#
# So the missing piece was never the price data. It was the symbol list.
# =====

```

```

FUNCTION build_pit_membership()
# ONE ROW PER SYMBOL PER SNAPSHOT. This is the listing record.
membership = {}
FOR f IN all_finra_files(2021-01 .. today):      # 131 snapshots
    symbols = read_column(f, "symbolCode")
    FOR s IN symbols:
        membership[s].APPEND({settle: f.date,
                               usable_from: f.date + SI_LAG_DAYS})
RETURN membership

FUNCTION listed_on(symbol s, day t)
# was s a listed NMS security at time t, as known at time t?
snap = last snapshot WHERE snapshot.usable_from <= t
RETURN s IN membership_at(snap)

FUNCTION build_full_symbol_set()
ever = union of all symbols across all snapshots
# filter to plausible common stock. The FINRA file includes OTC, warrants,
# units and preferred, none of which belong in this universe.
keep = []
FOR s IN ever:
    IF NOT s.isalpha() OR len(s) > 5:          CONTINUE
    IF s ENDS WITH "W" AND len(s) == 5:        CONTINUE # warrant
    IF s ENDS WITH "U" AND len(s) == 5:        CONTINUE # unit
    IF s ENDS WITH "R" AND len(s) == 5:        CONTINUE # right
    meta = alpaca_assets_all().get(s)          # may be null for purged symbols
    IF meta IS NOT null AND meta.exchange NOT IN
        {NYSE, NASDAQ, AMEX, ARCA, BATS}:      CONTINUE
    IF meta IS NOT null AND name_matches_fund_pattern(meta.name): CONTINUE
    keep.APPEND(s)
RETURN keep
# A symbol with NO metadata is KEPT, not dropped. Absence from the assets
# endpoint is exactly the condition this module exists to handle. Dropping
# it would reintroduce the bias this module removes.

FUNCTION fetch_all_bars_including_dead(symbols)
# the data endpoint does not care whether a symbol still trades
RETURN fetch_bars(symbols, "1Day", "2016-01-01", "split")
# AND the raw panel, per Module 1
# Symbols with no bars returned are recorded in a coverage report, never
# silently dropped. See the assertion below.

# ---- THE UNIVERSE FUNCTION THAT REPLACES LAYER 0 MEMBERSHIP -----
FUNCTION eligible_at(day t)
RETURN {s FOR s IN full_symbol_set
        WHERE listed_on(s, t)           # existed AND was known to exist
        AND has_bar(s, t)               # actually traded that day
        AND tradable(s, t)}             # Module 5, unchanged

# ---- ASSERTIONS -----
ASSERT for every test day t, eligible_at(t) contains symbols that were later
delisted. If it contains none, the membership join is broken and the
run is a survivor scan again.
ASSERT count(eligible_at(t)) is roughly stable across t. A universe that
grows monotonically toward the present is the signature of this bias.
ASSERT BBBY IN eligible_at(2021-06-01) AND BBBY NOT IN eligible_at(2024-06-01)
# a cheap, concrete canary for the whole module

FUNCTION coverage_report()
FOR each snapshot date:
    PRINT symbols in FINRA, symbols passing the filter,
           symbols with Alpaca bars, coverage ratio
    # A sample of 100 delisted symbols returned bars for 24. That sample
    # included warrants and OTC-only names, which Alpaca does not cover and
    # which the filter above removes. Measure coverage AFTER filtering before
    # claiming the bias is closed. If coverage is materially below 100% for
    # exchange-listed common stock, the residual gap must be reported as a
    # remaining bias, not assumed away.

# ---- WHAT THIS DOES NOT FIX -----
# 1. Before 2021. FINRA biweekly coverage starts 2021-01. The 2016 to 2020
#    warm-up window is still a survivor list. That window is used only to
#    establish all-time highs, not to trade, so the damage is bounded, but a
#    survivor's all-time high is not the same as the true one.
# 2. Symbols delisted BETWEEN two snapshots and never appearing in either.
#    A company listed and delisted inside one two-week window is invisible.
# 3. Alpaca bar coverage for delisted names is not guaranteed to be complete.
#    Measure it, report it, do not assume it.
# 4. Ticker reuse. A symbol freed by one delisting is often reassigned. The
#    membership record must be keyed on (symbol, date range), never on symbol
#    alone, or two different companies merge into one price series. This is
#    the same defect gate_2 catches downstream.

```

Every number reported before this module was built was measured on a survivor universe. Re-run the walk-forward with Module 22 in place before comparing against the reference values in Module 9. Those values will move, and they will move downward.

24 Module 23. Main driver

The preceding modules define 44 functions and never call any of them from a top level. This module is the only entry point. It answers what runs, in what order, and at what time of day.

```
MODULE main

# =====
# ONE CODE PATH SERVES BOTH BACKTEST AND LIVE.
# If the backtest and the live scanner are two programs, the backtest is not a
# forecast of the live system. Module 20 stage 2 verifies that claim
# empirically. This module makes it true by construction. The only permitted
# difference is where the clock comes from and where the fills come from.
# =====

FUNCTION one_trading_day(day t, mode, W, TH, state)
    # mode is "backtest" or "live". Nothing below branches on mode except the
    # two lines explicitly marked.

    # ---- 06:00 ET. Ingest. Every row lands with a publication date. ----
    FOR source IN [price_bars, symbol_master, short_interest, social,
                    options, borrow]:
        safe_fetch(source, fetcher[source], t)          # Module 17
        daily_health_report()                            # Module 17
    # gate_1 runs inside every join the fetchers perform.

    # ---- 08:00 ET. Layer 0 and Layer 1. Data through t-1 ONLY. ----
    universe = build_universe(t, W, TH)                 # Module 6 via 22
    on_universe_rebuild(universe, state.open_positions) # Module 21
    universe_churn_report()                             # Module 21

    # ---- 09:30 to 16:00 ET. Manage what is already open. ----
    manage_positions(t)                                # Modules 8, 14, 16

    # ---- 09:30 ET. Fill YESTERDAY's queue at TODAY's open. ----
    # The queue was built at yesterday's close. This is the t+1 entry that
    # Module 8 requires, and the reason expectancy is not overstated by 38%.
    IF mode == "live": submit_entry(s, t) FOR s IN state.queue # M19
    IF mode == "backtest": place_entries(state.queue, t-1, state.equity) # M8
    state.queue = []

    # ---- 16:00 ET. Layer 2. The ONLY step that reads day t. ----
    state.queue = scan_triggers(universe, t)             # Module 7

    # ---- 16:15 ET. LLM overlay, surviving names only. ----
    IF LLM_ENABLED:
        FOR (score, s) IN first MAX_POS entries of state.queue:
            llm_context(s, t, texts_for(s, t))          # Module 4, cached
        # Runs AFTER the queue exists, so it can never ADD a name. It may only
        # reorder names that were already selected deterministically.

    # ---- live only. Reconcile every assumption against reality. ----
    IF mode == "live": reconcile(t)                     # Module 19
    RETURN state

FUNCTION simulate(start, end, W, TH)
    # The function Module 9 calls. A loop over the SAME one_trading_day used
    # live. Not a second implementation.
    state = {equity: CAPITAL, open_positions: [], queue: [], trades: []}
    FOR t IN trading_days(start, end):
        state = one_trading_day(t, "backtest", W, TH, state)
    RETURN report(state.trades, daily_mark_to_market(state), spy_benchmark)

FUNCTION main()
    load_secrets()                                     # Module 18

    # ---- ONE TIME. Build the point-in-time universe. ----
    membership = build_pit_membership()                 # Module 22
    symbols = build_full_symbol_set()                   # Module 22
    resumable_fetch(symbols, "bars.ckpt",
```

```

        fetch_all_bars_including_dead)      # Modules 18, 22
coverage_report()                          # Module 22
ASSERT BBBY IN    eligible_at(2021-06-01)
ASSERT BBBY NOT IN eligible_at(2024-06-01)  # survivorship canary
build_blocklist(all_docs)                  # Module 2, gate 3

# ---- MEASURE BEFORE MODELLING. ----
ic = report_ic(all_features, universe = NEUTRAL)  # Module 10
FOR f IN all_features: gate_5_feature_sign(f, 20) # Module 2
# No feature enters the composite until its sign is measured HERE.
# Skipping this step is how social level was added with a positive weight
# against a measured IC of -0.054.

# ---- THE EXPERIMENT. ----
results = run_walkforward()                 # Module 9
IF NOT accept(results):                     # Module 9
    PRINT "does not clear SPY at 0.95. Stop. Do not proceed."
    HALT

# ---- ONLY IF ACCEPTED. ----
validate_source_parity()                    # Module 20
# then stages 2, 3 and 4 of Module 20, in order, skipping none

# ---- MODULE INDEX -----
# 0 constants          9 walk-forward driver  18 operations
# 1 ingest             10 evaluation           19 orders
# 2 quality gates      11 missing data        20 parity and staging
# 3 features           12 failure modes       21 universe churn
# 4 llm overlay        13 trending            22 point-in-time listing
# 5 tradability        14 halts and LULD      23 this module
# 6 universe           15 concentration
# 7 trigger            16 corporate actions
# 8 execution          17 degradation
#
# BUILD ORDER, which differs from the reading order:
# 0, 22, 1, 2, 3, 5, 6, 7, 8, 9, 10.
# Modules 4 and 13 through 21 are added afterwards, each behind a flag, each
# measured on its own before it is permitted to change a decision.
END MODULE

```